

COURSE NAME

SOFTWARE QUALITY
AND TESTING

CSC 4133

(UNDERGRADUATE)

CHAPTER 10

INTEGRATION TESTING

SYSTEM INTEGRATION TESTING

- ❑ Large, complex software products can go through several iterations of **build-and-test cycles** before they are **fully integrated**
- ❑ System integration testing (SIT) is a systematic technique for **assembling a software system** while conducting tests to **uncover errors associated with interfacing the modules**
- ❑ The objective is to build a “**working**” version of the system by:
 - Putting modules together in an **incremental** manner after conducting **unit testing on module**
 - Ensuring that the additional modules work as expected without disturbing the functionalities of the modules already put together
- ❑ **Integration testing is said to be complete when**
 - The system is fully integrated together
 - All the test cases have been executed
 - All the severe and moderate defects found have been fixed

SYSTEM INTEGRATION TESTING

□ Advantages of System Integration Testing

- Defects are detected early, It is easier to fix defects detected earlier
- Get earlier feedback on the acceptability of the individual modules and on the overall system
- Scheduling of defect fixes is flexible, and it can overlap with development process

□ “If all the unit-tested modules work individually, why can these modules not work when put together?”

- The problem arises when we “put modules together” because of interface errors
- Interface errors are those that are associated with structures existing outside the local environment of a module, but which the module uses

TYPES OF INTERFACES

❑ Procedure call interface (function call):

- A procedure in one module calls a procedure in another module. The caller passes on control to the called module. The caller can pass data to the called procedure, and the called procedure can pass data to the caller while returning control back to the caller.

❑ Shared memory interface (global variable):

- A block of memory is shared between two modules. The memory block may be allocated by one of the two modules or a third module. Data are written into the memory block by one module and are read from the block by the other.

❑ Message passing interface (interaction):

- One module prepares a message by initializing the fields of a data structure and sending the message to another module. This form of module interaction is common in client-server-based systems and web-based systems.

MODULARITY OF SIT

- ❑ Integration testing includes both **white-box** and **black-box** testing approaches.
- ❑ Various levels of granularities in SIT are :
 - ❖ **Intra-system testing**
 - constitutes **low-level integration testing** with the objective of combining the modules together to build a **cohesive system**
 - ❖ **Inter-system testing**
 - a **high-level testing** phase which requires **interfacing independently tested systems**
 - ❖ **Pair-wise testing**
 - only **two interconnected systems** in an overall system are tested at a time.
 - The purpose is to ensure that **two systems under consideration can function together**, assuming that the other systems within the overall environment behave as expected

SYSTEM INTEGRATION TECHNIQUES: INCREMENTAL INTEGRATION

- ❑ Integration testing is conducted in an incremental manner as a series of test cycles
- ❑ In each test cycle, a few more modules are integrated with an existing and tested to generate a larger build
- ❑ Key idea is to complete one cycle of testing, let the developers fix all the defects found, and continue the next cycle of testing.
- ❑ **Advantages:**
 - The incremental approach has the advantage that the defects are found early in a smaller assembly when it is relatively easy to detect the cause.
- ❑ **Disadvantages:**
 - A disadvantage is that it can be time-consuming since stubs and drivers have to be developed and used in the test.

SYSTEM INTEGRATION TECHNIQUES: TOP-DOWN INTEGRATION

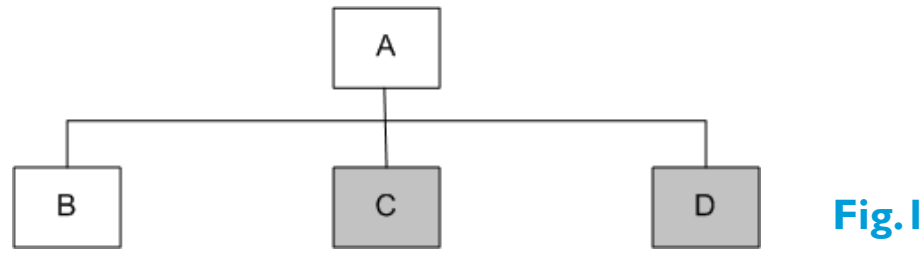


Fig.1

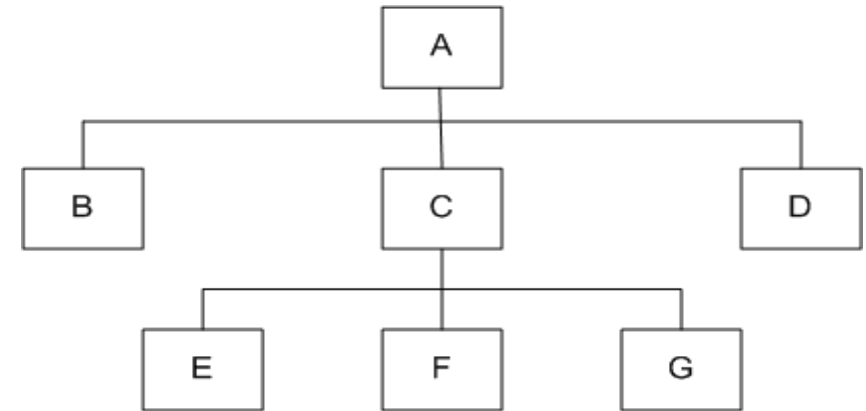


Fig.2

- Module A has been decomposed into modules B, C, and D
- Modules B, D, E, F, G are terminal modules (fig.2)
- First integrate modules A and B using stubs C and D (represented by grey boxes)
- Next stub D has been replaced with its actual instance D
- Two kinds of tests are performed:
 - Test the interface between A and D
 - Regression tests to look for interface defects between A and B in the presence of module D

SYSTEM INTEGRATION TECHNIQUES: TOP-DOWN INTEGRATION

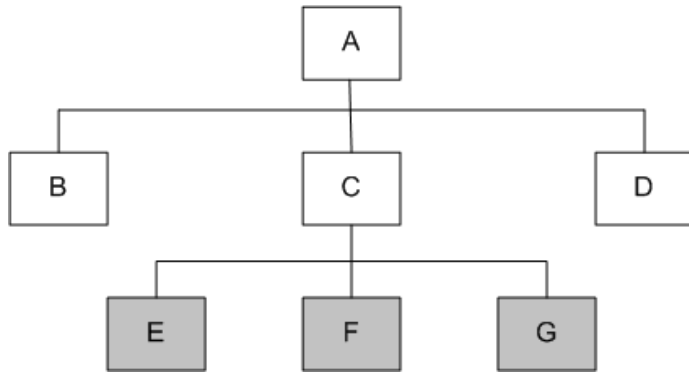


Fig1: Top-down integration of modules A, B, D and C

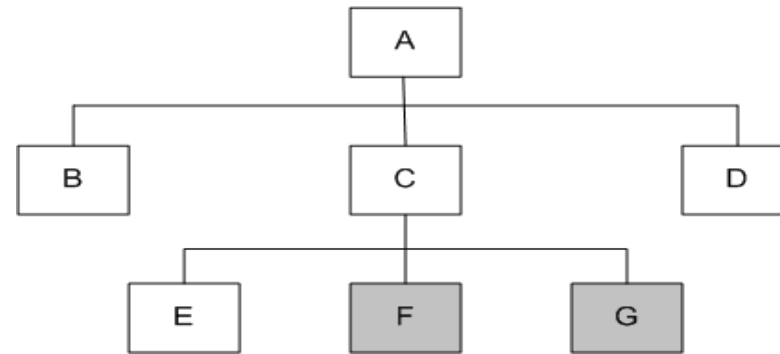


Fig2: Top-down integration of modules A, B, C, D and E

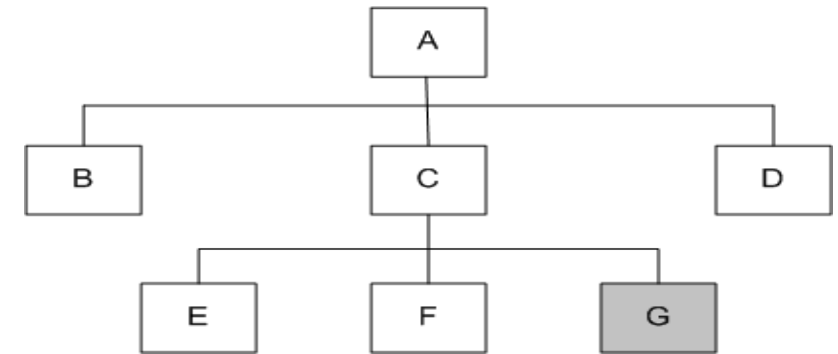


Fig3: Top-down integration of modules A, B, C, D, E and F

- Stub C has been replaced with the actual module C, and new stubs E, F, and G
- Perform tests as follows:
 - first, test the interface between A and C;
 - second, test the combined modules A, B, and D in the presence of C

SYSTEM INTEGRATION TECHNIQUES: TOP-DOWN INTEGRATION

Advantages

- SIT engineers continually observe system-level functions as the integration process continues
- Isolation of interface errors becomes easier because of the incremental nature of the top-down integration
- Test cases designed to test the integration of a module M are reused during the regression tests performed after integrating other modules

Disadvantages

- Basic functionality is tested at the end of cycle
- It may not be possible to observe meaningful system functions because of an absence of lower level modules and the presence of stubs.
- Test case selection and stub design become increasingly difficult when stubs lie far away from the top-level module.

SYSTEM INTEGRATION TECHNIQUES: **BOTTOM-UP INTEGRATION**

- System integration begins with the integration of lowest level modules
- A module is said to be at the lowest level if it does not invoke another module
- It is assumed that all the modules have been individually tested before
- To integrate a set of lower level modules, need to construct a test driver module that invokes the modules to be integrated
- Once the integration of a desired group of lower level modules is found to be satisfactory, the driver is replaced with the actual module and one more test driver is used to integrate more modules with the set of modules already integrated.
- The process of bottom-up integration continues until all the modules have been integrated
- Advisable when the project is unique in nature

SYSTEM INTEGRATION TECHNIQUES: BOTTOM-UP INTEGRATION

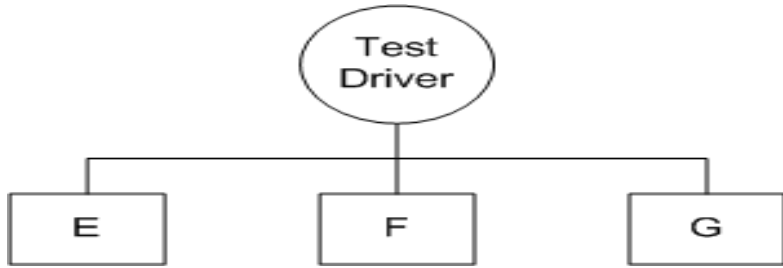
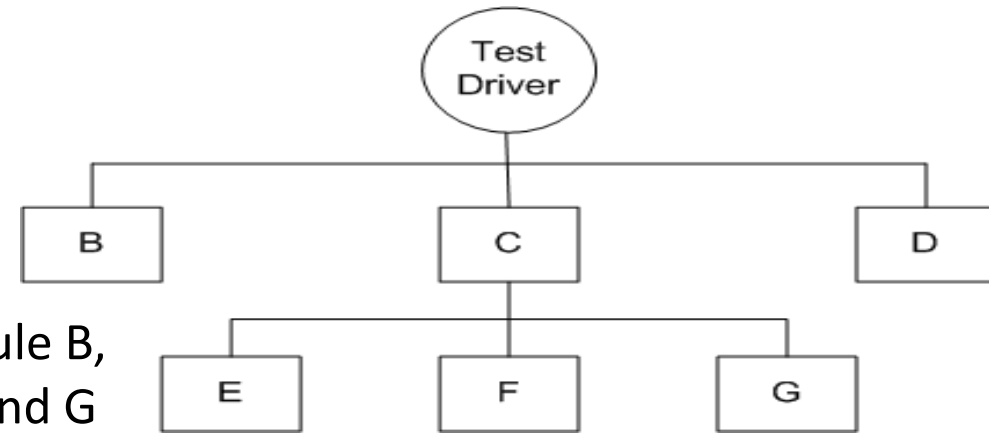


Fig 1: Bottom-up integration of module E, F, and G

Fig 2: Bottom-up integration of module B, C, and D with F, F, and G



- We design a test driver to integrate lowest-level modules E, F, and G
- The test driver simulates module C to integrate E, F, and G in a limited way.
- The test driver (C) is replaced with actual module
- A new test driver is used for more modules such as B and D are integrated
- The new test driver mimics the behavior of module A, finally
- The test driver is replaced with module A and further test are performed after the testers are satisfied

SYSTEM INTEGRATION TECHNIQUES: BOTTOM-UP INTEGRATION

Advantages

- If the low-level modules and their combined functions are often invoked by other modules, then it is more useful to test them first so that meaningful effective integration of other modules can be done (useful when the project is novel)

Disadvantages

- Discovery of major flaws in system design may not be possible until the top-level modules have been integrated
- Test engineers can not observe system-level functions from a partly integrated system
- We can catch the Key interface defects at the end of cycle

SYSTEM INTEGRATION TECHNIQUES: BIG BANG

- First all the modules are individually tested. Next, all those modules are put together to construct the entire system which is tested as a whole
- Sometimes developers use this approach for integrating small systems. However, for large systems, this approach is not recommended

Advantage

- Big Bang testing has the advantage that everything is finished before integration testing starts

Disadvantages

- The major disadvantage is that in general it is time consuming and difficult to trace the cause of failures because of this late integration

SYSTEM INTEGRATION TECHNIQUES: SANDWICH

- A system is integrated by using a mix of top-down, bottom-up, and big-bang approaches
- A hierarchical system is viewed as consisting of three layers
- The bottom-up approach is applied to integrate the modules in the bottom-layer
- The top layer modules are integrated by using top-down approach
- The middle layer is integrated by using the big-bang approach after the top and the bottom layers have been integrated

TEST PLAN FOR SYSTEM INTEGRATION

- System integration requires a controlled execution environment, much communication between the developers & the test engineers.
- Integrating a large system is a challenging task, which is handled with much planning in the form of developing a SIT plan.
- A useful framework for preparing an SIT plan can be used.

1. Scope of Testing
2. Structure of the Integration Levels a. Integration test phases b. Modules or subsystems to be integrated in each phase c. Building process and schedule in each phase d. Environment to be set up and resources required in each phase
3. Criteria for Each Integration Test Phase <i>n</i> a. Entry criteria b. Exit criteria c. Integration Techniques to be used d. Test configuration set-up
4. Test Specification for Each Integration Test Phase a. Test case id# b. Input data c. Initial condition d. Expected results e. Test procedure - How to execute this test? - How to capture and interpret the results?
5. Actual Test Results for Each Integration Test Phase
6. References
7. Appendix

TEST PLAN FOR SYSTEM INTEGRATION

Entry Criteria

Software functional and design specifications must be written, reviewed and approved.

Code reviewed and approved.

Unit test plan for each module is written, reviewed, and executed.

100% unit tests passed.

All the check-in request form must be completed, submitted, and approved.

Hardware design specification written, reviewed, and approved.

Hardware design verification test written, reviewed, and executed.

100% design verification tests passed.

Hardware/software integration test plan written, reviewed, and executed.

100% hardware/software integration tests passed.

TEST PLAN FOR SYSTEM INTEGRATION

Exit Criteria

All code completed and frozen and no more modules to be integrated.

100% system integration tests passed.

No major defect is outstanding.

All the moderate defects found in SIT phase have been fixed and re-tested.

Not more than 25 minor defects are outstanding.

Two weeks system uptime in system integration test environment without any anomalies, i.e., crashes.

System integration tests results are documented.

CATEGORIES OF SYSTEM INTEGRATION TEST

- **Interface integrity:** internal and external interfaces are tested as each module is integrated
- **Functional validity:** test are designed to uncover functional errors in each module after it is integrated with the system.
- **End-to-end validity:** tests are performed to ensure that a completely integrated system works together from end-to-end.
- **Pair-wise validity:** tests are performed to ensure that any two systems work properly when connected together by a network
- **Interface stress:** tests are performed to ensure that the interfaces can sustain the load
- **System endurance:** tests are performed to ensure that the integrated system stay up for weeks without any crashes

OFF-THE-SHELF COMPONENT INTEGRATION

- Instead of developing a software component from scratch, organizations occasionally purchase off-the-shelf (OTS) components from third-party vendors and integrate them with their own components.
- OTS components produced by the vendor organizations are known as commercial off-the-shelf (COTS) components.

Advantages of COTS component integration

- Less time consuming
- Least costly
- More reliable

OFF-THE-SHELF COMPONENT INTEGRATION

Challenges of COTS component integration

- A major issue that can arise while integrating different components is mismatches among code pieces developed by different parties usually unaware of each other.
- Integration not always trivial; incompatibilities among vendors
- The buyer has no access to the source code
- Dependence on vendor – the vendor controls its development
- Stability of the vendor – vendors can go out of business, can be purchased by other companies or completely drop support for a product

OFF-THE-SHELF COMPONENT TESTING

- ❑ Black-box component testing
 - used to determine the quality of the component
- ❑ System-level fault injection testing
 - used to determine how well a system will tolerate a failing OTS component
- ❑ Operational system testing
 - used to determine the tolerance of a software system when the OTS component is functioning correctly. Operational system test is conducted to ensure that an OTS component is a good match for the system.

REFERENCES

- ❑ Software Testing And Quality Assurance – Theory and Practice - Kshirasagar Naik & Priyadarshi Tripathy
- ❑ Software Quality Engineering: Testing, Quality Assurance and Quantifiable Improvement - Jeff Tian